

Branching and looping

EMATM0048- Teaching Session 2

Presented by: Dr. Ayush Joshi



Last session:

- Syntax and semantics
- Python elements
- Objects and simple operations
- Expressions, variables and values

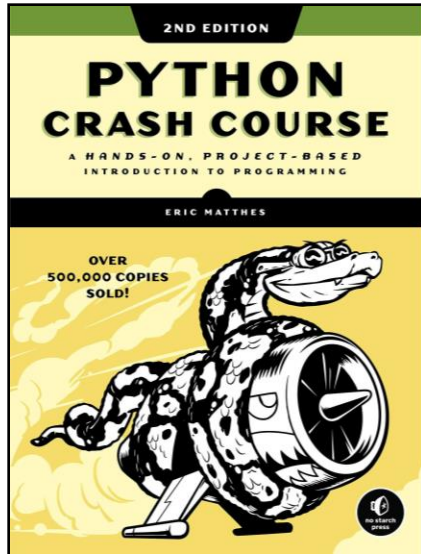
Session's contents

- String type
- Branching and conditionals
- Iteration and loops

Readings

Chapter 5

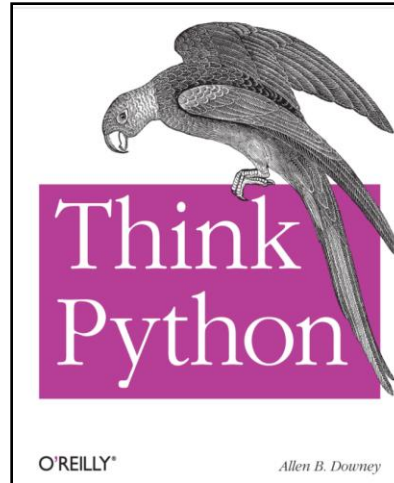
Chapter 7: (7.1)



Chapter 5: (5.4)

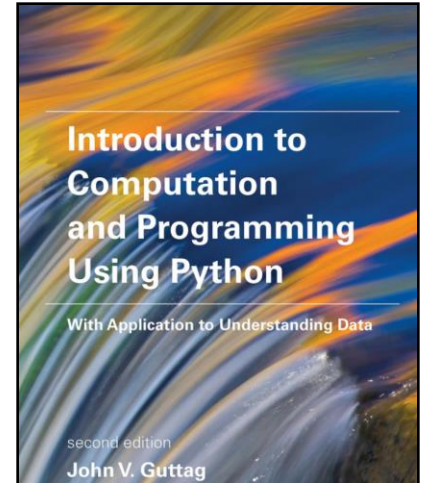
Chapter 7: (7.3), (7.4)

Chapter 8: (8.1), (8.5)



Chapter 2.2

Chapter 2.4



String type

- A *string* is simply a series of characters (letters, special characters, spaces, digits).
- How to concatenate strings?
- More about emojis at <https://www.geeksforgeeks.org/python-program-to-print-emojis/>

```
x= "Alpaca is cute "  
print (x)  
print("\N{winking face}")
```

```
print (x + "\N{winking face}")
```

Strings and mathematical operators

- The **+** operator performs string concatenation

```
x= "Hello"  
y= "there"  
print (x + y)  
print (x,y)
```

- The ***** operator also works on strings; it performs repetition. For example, 'Spam'*3 is 'SpamSpamSpam'.
 - If one of the values is a string, the other has to be an integer.

String type

Apply cast and try again

```
x= "Alpaca is cute "  
print (x + 3)
```

- You can only concatenate strings with strings! **Solution:** Casting
- Strings are Python objects that have: data, methods and functions
- Access this information by object_name.do_something()

```
strName.upper()
```

String methods

```
strName.upper()
```

```
strName.lower()
```

```
strName.title()
```

```
strName.rstrip()
```

```
strName.lstrip()
```

```
strName.strip()
```

```
len(strName)>> number of characters
```



Try with
strName= "Ada Lovelace"

Indexing Strings

- A string is a sequence.

var =	0	1	2	3	4	5
	P	Y	T	H	O	N

```
>>var[0]
'P'
>>var[5]
'N'
>>var[6] ??
```

Slicing Strings

- String can be sliced using [Start: Stop: Step]
- Step is optional, with default value of 1
 - `s[2:5]` >> `s[2:5:1]`
- You can also omit the numbers and leave just colons

```
>>s[:2]  #same as s[0:2]  
>>s[3:]  #same as [3:end]  
>>s[::]  #same as [0:end:1]
```

Try this

S

Indexing Strings

var[start:stop:step]

Inclusive

Not-Inclusive

```
>>var[0:3] # same as var[:3]
'PYT'
>>var[4:]
'ON'
>>var[:5:2] ??
```

PTO

0	1	2	3	4	5
P	Y	T	H	O	N

Indexing Strings

- A string is a sequence.

var =

0	1	2	3	4	5
P	Y	T	H	O	N
-6	-5	-4	-3	-2	-1

```
>>var[-2]
'0'
>>var[-4:]
'THON'
```

*# Your turn! How
would we reverse the
string using this
notation?*

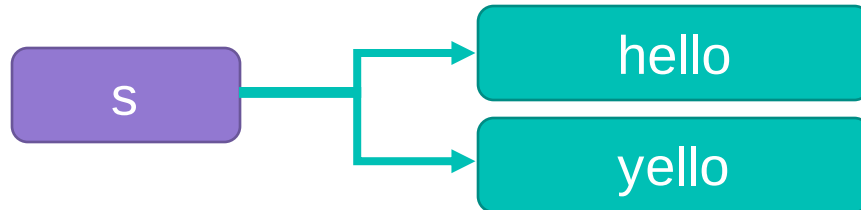
var[::-1]

String type

- Strings are “**immutable**” –cannot be modified

```
>> s = "hello"  
>> s[0] = 'y'    #gives an error  
>> #how can you replace first letter with y?  
>> #is it a new object?
```

```
>>s = 'y'+s[1:]
```



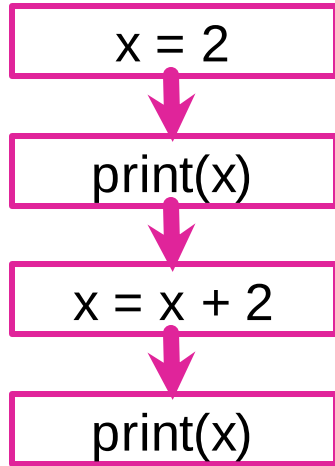
Session's contents

- String type
- Branching and conditionals
- Iteration and loops

Program Flow

- Like a recipe, a program is a sequence of steps to be done in order.
- Some steps are conditional - they may be skipped. (If statements)
- Sometimes a step or group of steps is to be repeated. (loops)
- Sometimes we store a set of steps to be used over and over as needed several places throughout the program. (functions)

Sequential Steps



```
x = 2  
print(x)  
x = x + 2  
print(x)
```

Output

2
4

When a program is running, it flows from one step to the next. As programmers, we set up "paths" for the program to follow.

Branching and conditional

- Programming often involves examining a set of conditions and deciding which action to take based on those conditions.

```
if <condition>:  
    <expression>  
    <expression>
```

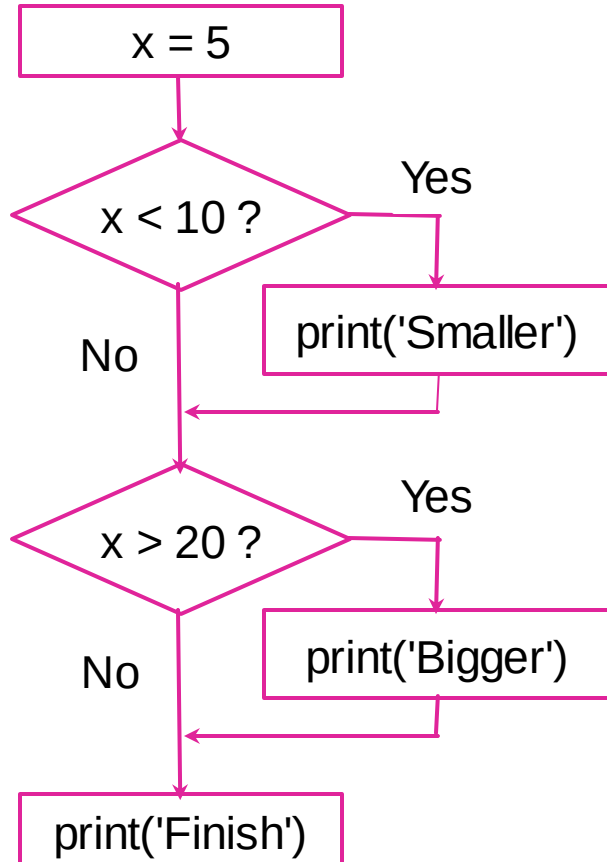
- **<condition>** has a value **True** or **False**
 - Evaluate expressions in that block if **<condition>** is **True**

If statements:

- No parentheses necessary around the condition
- Colon at the end (no braces!)
- Use four spaces (or one tab) for indentation

```
if <condition>:  
    <expression>  
    <expression>
```

Conditional Steps



```
x = 5
if x < 10:
    print('Smaller')
if x > 20:
    print('Bigger')
print('Finish')
```

Output

Smaller
Finish

Chained Conditional (`elif`)

- Sometimes there are more than two possibilities and we need more than two branches.
- `elif` is an abbreviation of "else if".
- Again, exactly one branch will run. There is no limit on the number of `elif` statements.
- If there is an `else` clause, it has to be at the end, but there doesn't have to be one.

Nested conditionals

- One conditional can also be nested within another.

```
if x == y:
    print('x and y are equal')
else:
    if x < y:
        print('x is less than y')
    else:
        print('x is greater than y')
```

```
if x == y:
    print("x and y are equal")
    if y != 0:
        print("therefore, x / y is", x/y)
elif x < y:
    print("x is smaller")
elif x > y:
    print("y is smaller")
print("done!")
```

Control Flow -Branching

```
if <condition>:  
    <expression>  
    <expression>  
    ...
```

```
if <condition>:  
    <expression>  
    <expression>  
else:  
    <expression>  
    <expression>
```

```
if <condition>:  
    <expression>  
    <expression>  
    ...  
elif <condition>:  
    <expression>  
    <expression>  
    ...  
else:  
    <expression>  
    <expression>  
    ...
```

Python Indentation

- **Indentation** is semantically meaningful in Python
- Python is unusual in using indentation this way.
 - Most other programming languages use some sort of bracketing symbols to delineate blocks of code
- An advantage of the Python approach is that it ensures that the visual structure of a program is an accurate representation of the semantic structure of that program.

Session's contents

- String type
- Branching and conditionals
- Iteration and loops

Control flow: `while` Loops

- `<condition>` evaluates to a Boolean
- if `<condition>` is True, do all the steps inside the `while` code block
- check `<condition>` again
- repeat until `<condition>` is False

The body of the loop should change the value of one or more variables so that the condition. Why?

```
while <condition>:  
    <expression>  
    <expression>  
    ...
```

Your turn!

Write a program that prints the following
statement until you turn left

```
You are Lost in the Forest. Go left  
or right?
```

Solution!

```
n = input("You're in the Lost Forest. Go left or right? ")
while n == "right":
    n = input("You're in the Lost Forest. Go left or right? ")
print("You got out of the Lost Forest!")
```

Control flow: `for` Loops

- Each time through the loop, `<item>` takes a value
- No loop counter! Instead, the item variable takes on the value of sequential elements of the iterable.
- Iterables can be strings, lists, or one of any one of a number of other data types

```
for <item> in iterable:  
    <expression>  
    <expression>  
    ...
```

What is range?

```
range(start, stop[, step])
```

- Range is a built in function generates an iterable over a range of numbers.
- Commonly used with loops
- default values are start = 0 and step = 1 and optional loop until value is stop -1
- Same rules as string slicing! `start` is inclusive, `stop` is exclusive

```
>>range(5,40,10)  
>>range(40,5,-10)  
>>range(5)
```

Control flow: `for` Loops

- iterate through numbers in a sequence

```
n = 0
while n < 5 :
    print(n)
    n+=1
```

```
for n in range(5):
    print(n)
```

What is the difference?

- Does changing the value of x inside the loop affects the number of iterations?

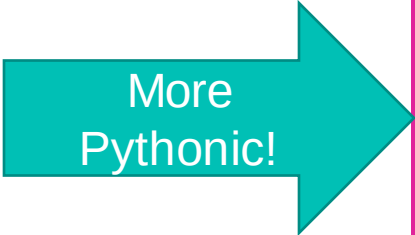
```
x = 4
for i in range(0, x):
    print(i)
```

```
x = 4
for i in range(0, x):
    print(i)
    x = 5
```

Strings and loops

```
s = 'A random text'
i=0
while i<len(s):
    char= s[i]
    if(char=='a'):
        print('there is an a')
    i+=1
```

```
s = 'A random text'
for index in range(len(s)):
    if s[index]=='a':
        print('there is an a')
```



More
Pythonic!

```
s = 'A random text'
for ch in s:
    if ch== 'a':
        print('there is an a')
```

break Statement

- immediately exits whatever loop it is in
 - skips remaining expressions in code block
 - exits only innermost loop!

```
x = 6
while x: #x>0
    print(x)
    x -= 1
    if x == 3:
        break
```

```
while <condition_1>:
    while <condition_2>
        <expression_a>
        break
        <expression_b>
    <expression_c>
```

Continue **statement**

- continues to the next iteration of the loop

```
current_number = 0
while current_number < 10:
    current_number += 1
    if current_number % 2 == 0:
        continue
    print(current_number)
```

Takeaways

- String is a sequence of characters, can be sliced using indexes.
- Conditional blocks are expressed with if statements
- Loops represented by while and for loops
- Indentation is crucial in python in defining blocks
- Break and continue can be used to control the flow.

Next session:

- Compound types in Python
 - Lists
 - Tuples